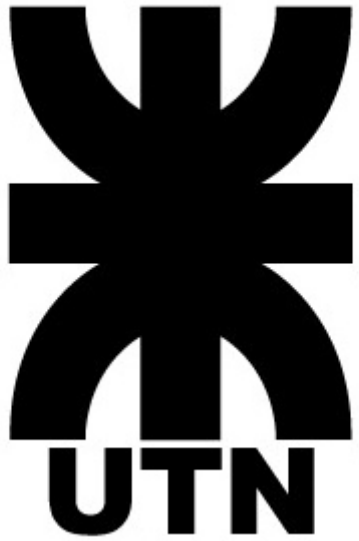


UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL TUCUMÁN

PRÁCTICA SUPERVISADA

INFORME FINAL

NUBE ACADÉMICA PRÁCTICA

Software de gestión y orquestación de servicios académicos

ALUMNO · Armas, Mauro

CARRERA · Ingeniería en Sistemas de Información

DOCENTE SUPERVISOR · Ing. Luis María Carriles

TUTORA INSTITUCIONAL · Mg. Ing. Silvana Carolina Marsiglia

CÁTEDRA · Virtualización y Consolidación de Servidores

14 de septiembre — 3 de noviembre de 2026 | 200 horas

San Miguel de Tucumán · Noviembre de 2026

Resumen **3**
Introducción y objetivos **3**
1. Contexto, entorno operacional y alcance **4**
1.1 Necesidad institucional 4
1.2 Entorno operacional 4
1.3 Usuarios y alcance del producto 4
1.4 Requisitos principales 5
2. Propuesta de solución y diseño **6**
2.1 Visión del producto 6
2.2 Arquitectura general 6
2.3 Gestión del ciclo de vida 8
2.4 Principios de diseño 9
2.5 Modelo de información y trazabilidad 9
3. Módulos y funcionalidades del software **10**
3.1 Identidad, usuarios y seguridad de acceso 10
3.2 Organización académica y catálogo de servicios 11
3.3 Pedidos, aprobaciones y calendario 13
3.4 Capacidad y orquestación 14
3.5 Aprovisionamiento y acceso a servicios 16
3.6 Aislamiento, supervisión y notificaciones 17
3.7 Experiencia de usuario y administración 19
4. Decisiones técnicas y criterios de calidad **20**
4.1 Tecnologías elegidas 20
4.2 Seguridad, recuperación y mantenibilidad 20
4.3 Evaluación de alternativas y compromisos 20
5. Validación del producto y resultados **22**
5.1 Estrategia de pruebas 22
5.2 Rendimiento bajo carga 22
5.3 Validación sobre infraestructura real 23
5.4 Alcance de las evidencias y pruebas pendientes 24
6. Problemas relevantes y mejoras incorporadas **26**
6.1 Ajustes de arquitectura y seguridad 26
6.2 Simplificación del despliegue de aplicaciones 26
6.3 Aprendizajes de la integración 26
7. Cumplimiento del plan de trabajo (200 horas) **27**
8. Conclusión y líneas de evolución **27**
8.1 Conclusión 27
8.2 Trabajo futuro 28
9. Referencias **28**

Resumen

La presente Práctica Profesional Supervisada (PPS) tuvo como finalidad diseñar, desarrollar y validar el software de gestión y orquestación de la Nube Académica Práctica (NAP) de la UTN-FRT. La solución busca que las cátedras puedan solicitar y utilizar recursos informáticos desde un portal web, sin administrar directamente la plataforma de virtualización. El trabajo individual documentado en este informe comprende la ingeniería del software; el clúster de servidores, las redes y el almacenamiento constituyen el entorno operativo sobre el cual se integra.

NAP ofrece un catálogo de servicios de infraestructura (IaaS), plataformas (PaaS) y aplicaciones (SaaS): entornos Linux, bases de datos MariaDB y aplicaciones web, tanto prediseñadas como provistas mediante imágenes de contenedor. Incluye autenticación y permisos por rol, pedidos sujetos a aprobación, asignación controlada de capacidad, aprovisionamiento automático, acceso seguro a los servicios, seguimiento de consumo y notificaciones.

La arquitectura separa la interacción con los usuarios, las reglas del negocio, la persistencia de información y la ejecución de tareas de infraestructura. Se adoptaron tecnologías ampliamente utilizadas —React, Python/FastAPI, PostgreSQL, Docker y Proxmox VE— para favorecer modularidad, mantenibilidad y adaptación a los recursos de una institución educativa. Según los registros de validación del proyecto, se ejecutaron más de 900 pruebas automatizadas, ensayos de rendimiento y pruebas reales en laboratorio. La integración avanzada con los cuatro nodos definitivos de la facultad permanece como actividad de puesta en producción. El resultado es un producto funcional y validado en escenarios representativos, con una base técnica preparada para su operación institucional.

Introducción y objetivos

Las prácticas de laboratorio de Ingeniería en Sistemas de Información requieren sistemas operativos, bases de datos y aplicaciones con características diferentes según la asignatura. Cuando cada cátedra resuelve estas necesidades por separado, se multiplican las instalaciones, los problemas de compatibilidad y las tareas manuales de soporte. La Nube Académica Práctica propone centralizar estos recursos y acercarlos a docentes mediante una experiencia de autoservicio gobernada por reglas institucionales.

El objetivo general de esta PPS fue desarrollar el software que convierte una infraestructura de virtualización existente en un servicio académico administrable. El portal debía permitir solicitar, aprobar, desplegar, consultar, operar y retirar servicios, manteniendo trazabilidad, seguridad y control sobre la capacidad disponible.

Como objetivos específicos se definieron: relevar las necesidades de los usuarios; diseñar una arquitectura modular; implementar los flujos de autenticación, catálogo, pedidos y aprovisionamiento; integrar el software con la infraestructura; incorporar seguimiento y medidas de protección; validar su comportamiento y documentar su utilización. La exposición se concentra en las decisiones de ingeniería y en el valor funcional del sistema, no en la descripción línea por línea de su implementación.

1. Contexto, entorno operacional y alcance

1.1 Necesidad institucional

Docentes y estudiantes necesitan entornos comparables entre sí, disponibles durante un período académico concreto y adecuados al contenido de cada materia. Una asignatura inicial puede requerir una herramienta web para diagramar; una materia de desarrollo necesita una base de datos accesible; y una materia avanzada puede precisar un servidor Linux administrable. La plataforma debe responder a estos casos sin exigir que el docente conozca conceptos de hipervisores, redes virtuales o políticas de almacenamiento.

La oportunidad detectada fue construir una capa de software que traduzca necesidades académicas —qué servicio se necesita, para qué cátedra, con cuántos recursos y durante cuánto tiempo— en operaciones de infraestructura controladas. Esa capa también debe evitar solicitudes que excedan la capacidad disponible y conservar un registro de las decisiones tomadas.

1.2 Entorno operacional

NAP funciona sobre un entorno de nube privada de la UTN-FRT compuesto por un clúster de virtualización Proxmox VE, conectividad de red administrada y almacenamiento centralizado mediante TrueNAS. Los servidores ofrecen recursos de procesamiento y memoria; la red permite comunicar a los usuarios con los servicios y separar los segmentos de gestión; el almacenamiento compartido aloja datos y volúmenes que necesitan persistencia. Estas capacidades son dependencias de operación del producto, no componentes desarrollados como parte de esta PPS individual.

El software se conecta con Proxmox mediante interfaces de administración y ejecuta las operaciones autorizadas sobre los nodos. La persistencia del propio portal se resuelve de manera independiente en PostgreSQL. Esta separación permite describir responsabilidades con claridad: el entorno físico y de virtualización aporta capacidad; NAP administra quién puede consumirla, cuándo se reserva, qué se despliega y cómo se supervisa.

Durante la validación se utilizaron tanto ambientes de pruebas controlados como hipervisores reales, incluido un laboratorio de dos nodos con almacenamiento compartido. La aplicación de las pruebas avanzadas sobre los cuatro servidores definitivos de la facultad quedó pendiente por restricciones operativas asociadas al equipamiento.

1.3 Usuarios y alcance del producto

El sistema contempla dos perfiles principales. El responsable de cátedra consulta el catálogo, solicita y administra únicamente los recursos vinculados con su asignatura. El administrador general mantiene la organización académica, los usuarios, el catálogo y las reglas de disponibilidad, evalúa pedidos y supervisa la infraestructura desde el portal.

El alcance funcional incluye servicios Linux accesibles por terminal (IaaS), bases de datos listas para conectarse (PaaS) y aplicaciones web publicadas de forma controlada (SaaS). NAP no pretende reemplazar al hipervisor ni administrar directamente toda la red institucional: integra esas capacidades mediante un flujo comprensible para los usuarios académicos.

El alcance temporal comprende el ciclo de vida completo de una solicitud: creación, evaluación, aprobación o rechazo, despliegue inmediato o programado, acceso, seguimiento, renovación y baja. Las acciones quedan sujetas al calendario lectivo y a la capacidad efectivamente disponible.

Desde el punto de vista operacional, la red de administración, el almacenamiento compartido y los nodos de cómputo se consideran dependencias de la plataforma. El software no sustituye sus herramientas de mantenimiento: consume sus capacidades a través de interfaces controladas y conserva un inventario lógico de los servicios que administra. Esta delimitación reduce la

confusión entre el producto desarrollado y los componentes físicos o de virtualización preexistentes.

Se definió además que la información de una cátedra debía permanecer segregada a lo largo de todo el recorrido: desde el pedido hasta los accesos, la telemetría y las notificaciones. Esa separación no podía limitarse a ocultar opciones en la interfaz; debía verificarse también al procesar cada solicitud en el servidor y al configurar el entorno de red.

1.4 Requisitos principales

• Usabilidad: solicitar un entorno mediante un formulario basado en propósito académico, tamaño y período, sin exponer parámetros internos del hipervisor.

• Control de acceso: separar los permisos de administración y cátedra, y evitar que una asignatura consulte los servicios o las credenciales de otra.

• Disponibilidad de capacidad: evaluar los recursos antes de aprobar y conservar su reserva hasta que se confirme la liberación.

• Automatización: ejecutar solicitudes aprobadas sin pasos manuales repetitivos y comunicar al usuario el estado de cada operación.

• Seguridad: proteger las credenciales, cifrar las conexiones cuando corresponda y aplicar aislamiento de red entre servicios.

• Trazabilidad y continuidad: registrar cambios relevantes, observar el estado real de las instancias y admitir recuperación ante fallos.

• Rendimiento: mantener respuestas ágiles del portal incluso con múltiples usuarios y un historial amplio de métricas.

Del requerimiento académico al criterio verificable

Para que el desarrollo no quedara reducido a un catálogo de pantallas, cada necesidad se tradujo en una regla observable. Por ejemplo, la solicitud de un entorno por parte de un docente no se considera satisfecha cuando el formulario se guarda, sino cuando el pedido queda asociado a una cátedra, una oferta y una vigencia; puede ser evaluado por administración; y su resultado se refleja en el estado que consulta el solicitante. Del mismo modo, el alta de una aplicación no equivale a su disponibilidad: deben existir recursos asignados, una instancia creada y una verificación positiva de su servicio.

El criterio de aislamiento tiene dos dimensiones complementarias. En la aplicación, el responsable de una cátedra no puede obtener pedidos, credenciales o métricas de otra, aunque manipule identificadores o direcciones de acceso. En la infraestructura, los contenedores no deben alcanzar por red los servicios de otras asignaturas ni los segmentos administrativos. Esta distinción fue importante porque una interfaz que oculta botones no constituye una barrera de seguridad: el control debe mantenerse en el servidor y en el plano de ejecución.

Los requisitos de capacidad también se formularon en términos verificables. Antes de aprobar una solicitud, el sistema debe evaluar CPU, memoria y almacenamiento del destino; considerar reservas existentes; y evitar que dos aprobaciones simultáneas consuman el mismo saldo. La prueba pertinente no consiste únicamente en observar un número en un panel, sino en provocar solicitudes concurrentes y verificar que el inventario, los compromisos persistidos y el estado del clúster continúen siendo consistentes.

En la planificación inicial se mencionaban cuotas por cátedra y plantillas estandarizadas. Durante el diseño se refinó la política: el catálogo expresa tamaños conocidos y la capacidad se compromete al aprobar cada pedido, sin inmovilizar por adelantado recursos que una asignatura quizá no utilice. Esta evolución no modifica el propósito del plan —asignar recursos de forma controlada—, pero adapta su mecanismo a una infraestructura compartida y limitada. El resultado permite explicar tanto la necesidad institucional como la decisión técnica que la resuelve.

Finalmente, se diferenciaron los criterios de aceptación de laboratorio y de operación institucional. Las pruebas unitarias y de integración verifican reglas internas; los ensayos sobre hipervisores reales comprueban la interacción con Proxmox y los servicios; y la puesta en producción exige además certificados, accesos institucionales, disponibilidad del clúster definitivo y usuarios de aceptación. Conservar esta separación evita presentar como completamente desplegado un sistema que todavía requiere tareas de transferencia.

2. Propuesta de solución y diseño

2.1 Visión del producto

NAP se diseñó como un portal de autogestión gobernada. Desde la perspectiva docente, el recorrido comienza en el catálogo y termina en un servicio disponible con instrucciones de acceso. Desde la perspectiva administrativa, el valor está en contar con una única herramienta para evaluar solicitudes, utilizar la capacidad física responsablemente y supervisar la operación. La propuesta no consiste en ofrecer toda la complejidad de una plataforma de nube comercial, sino en seleccionar las funciones que realmente necesita el contexto universitario.

La interfaz distingue las tareas de cada perfil y ofrece información de estado en términos comprensibles: pedido en revisión, aprobado, en despliegue, listo para utilizar o con un inconveniente que requiere atención. El detalle del aprovisionamiento queda encapsulado en el motor de orquestación.

El diseño del producto parte de dos necesidades distintas: para el docente importa obtener un entorno utilizable en pocos pasos; para el administrador importa decidir si esa solicitud resulta pertinente y puede atenderse con la capacidad institucional. Por ello, NAP no utiliza una política de autoservicio irrestricto. Implementa un mecanismo de aprobación que permite conservar criterios académicos sin obligar a los usuarios a intervenir sobre la consola técnica del hipervisor.

La experiencia se orienta al lenguaje de la institución: cátedra, propósito, cuatrimestre, oferta y vigencia. La traducción a unidades de CPU, memoria y almacenamiento se realiza dentro del sistema, mediante perfiles predefinidos y reglas de admisión. Esto permite reutilizar un mismo catálogo para asignaturas con necesidades distintas y evita crear una instancia para cada aplicación antes de que exista demanda real.

2.2 Arquitectura general

Se separaron cinco responsabilidades. El portal web concentra la experiencia de usuario; una API gestiona autenticación y reglas del negocio; una base de datos conserva solicitudes, servicios, credenciales protegidas e historial; un orquestador ejecuta operaciones prolongadas; y un mecanismo de acceso seguro conecta al usuario con las aplicaciones desplegadas. La infraestructura Proxmox y el almacenamiento permanecen como recursos externos administrados por estas capacidades.

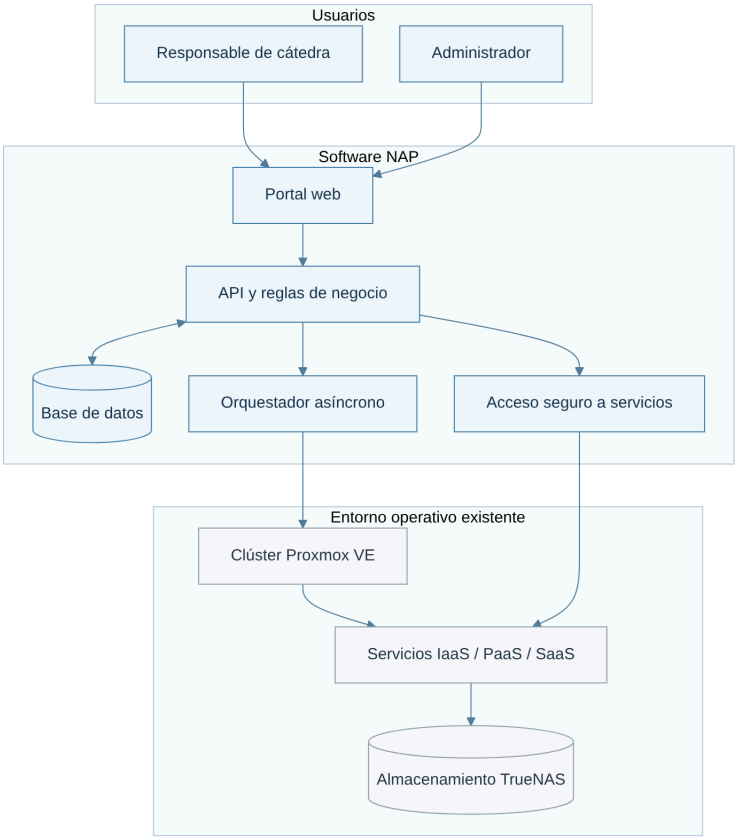


Figura 1. Arquitectura de alto nivel de NAP y su integración con el entorno operativo.

La capa de software concentra la lógica académica y las decisiones de aprovisionamiento; el clúster y el almacenamiento ejecutan y alojan los servicios.

Separar la atención de solicitudes de la ejecución de tareas permite mantener la interfaz disponible mientras se crea o configura un entorno. Asimismo, el componente de orquestación puede comprobar el resultado real de una operación antes de informar que terminó. Esta arquitectura reduce acoplamientos entre la experiencia docente y las particularidades del hipervisor.

El plano de control reúne las operaciones visibles para los usuarios: validaciones, aprobaciones, inventario, estados y políticas. El plano de ejecución se ocupa de las tareas de infraestructura que pueden durar segundos o minutos. La separación se implementa con una API REST y un proceso asíncrono que atiende una cola persistente de operaciones. La respuesta a una aprobación no equivale al éxito del despliegue: confirma que la solicitud fue admitida y queda registrada para su ejecución.

La persistencia relacional cumple un doble papel. Almacena la información de negocio y también coordina las tareas concurrentes. PostgreSQL permite resolver de forma transaccional la aprobación, reserva de capacidad y asignación de identidad del recurso, evitando que solicitudes simultáneas comprometan el mismo contenedor o excedan un saldo disponible. La operación en segundo plano conserva información sobre sus intentos y puede ser reconciliada con el estado real de Proxmox en caso de interrupción.

Para tareas dentro del nodo de virtualización se estableció un canal de ejecución acotado, en lugar de otorgar al portal acceso general a una terminal privilegiada. Así, el orquestador delega sólo acciones definidas y valida la respuesta de la infraestructura. Esta decisión favorece el aislamiento de responsabilidades y reduce el impacto potencial de un defecto en la capa web.

2.3 Gestión del ciclo de vida

El pedido constituye el punto de unión entre una necesidad académica y una asignación concreta de recursos. La aprobación administrativa comprueba viabilidad, reserva capacidad y autoriza el despliegue en la fecha prevista. Una vez creado el servicio, el docente obtiene el acceso correspondiente y puede realizar acciones permitidas durante su vigencia. Si requiere más tiempo, solicita una renovación sin perder necesariamente el entorno ni sus datos.

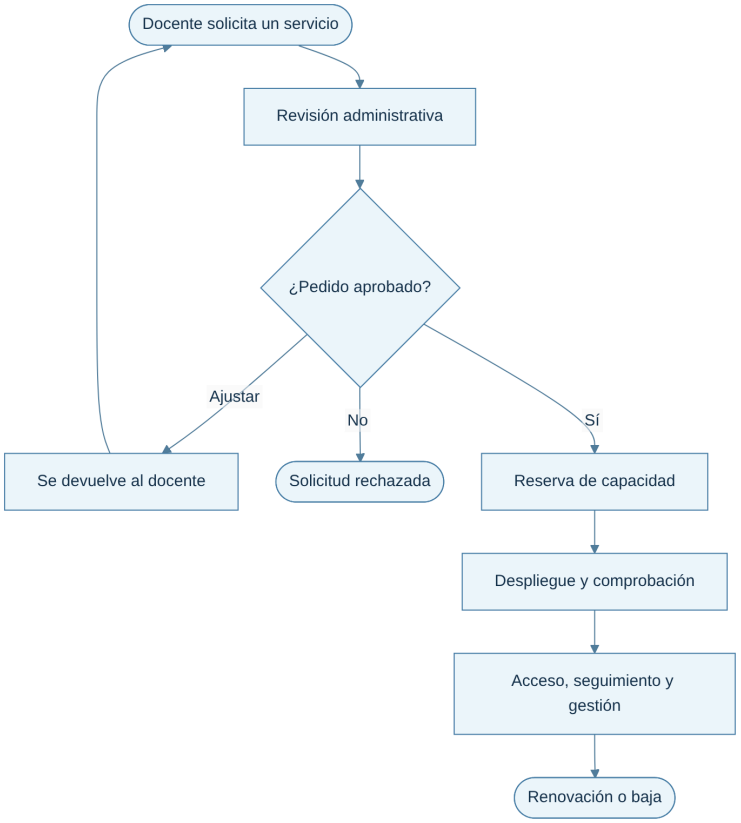


Figura 2. Recorrido funcional desde la solicitud académica hasta la renovación o baja.

El flujo incluye revisión administrativa y reserva previa al aprovisionamiento. Los pedidos que necesitan aclaraciones regresan al docente; los fallos de despliegue pueden tratarse mediante una operación de reparación.

Para mantener consistencia, NAP diferencia el estado administrativo del pedido, el estado operativo del contenedor y la disponibilidad funcional de la aplicación. Así, que un servidor esté encendido no significa automáticamente que una base de datos o una aplicación web ya esté preparada para recibir conexiones.

En una solución de infraestructura, los estados de negocio deben diferenciarse de las observaciones técnicas. Un pedido aprobado puede no estar desplegado si su inicio quedó programado. Un contenedor encendido puede alojar una aplicación que todavía está inicializando. Del mismo modo, una comunicación interrumpida durante una baja no demuestra que el disco asociado haya sido eliminado. La plataforma representa estas situaciones por separado y evita mostrar como ‘disponible’ algo que no pudo verificar.

El ciclo de vida contempla solicitudes que requieren ajustes, rechazos justificados, aprobación, despliegue, servicio activo, error recuperable, suspensión, renovación y baja definitiva. Estas transiciones se registran y generan notificaciones cuando afectan la actividad docente. La trazabilidad facilita tanto la gestión cotidiana como la explicación de incidentes ante el área técnica.

2.4 Principios de diseño

• Modularidad: las funciones de usuarios, catálogo, pedidos, infraestructura y observabilidad se gestionan como responsabilidades separadas.

• Mínimo privilegio: los usuarios interactúan con funciones autorizadas y el software restringe las operaciones que delega sobre el hipervisor.

• Consistencia de recursos: una aprobación no puede comprometer capacidad que ya fue asignada a otra solicitud, y una operación incierta no libera recursos prematuramente.

• Seguridad por defecto: el acceso a los servicios requiere los permisos, las credenciales y las conexiones establecidos para cada modalidad.

• Trazabilidad: las acciones relevantes y sus resultados quedan registradas para poder analizar incidentes y recuperar el estado del sistema.

2.5 Modelo de información y trazabilidad

En términos conceptuales, el modelo de datos se organiza alrededor de usuarios, cátedras, ofertas, pedidos, servicios e identidades de infraestructura. Una cátedra origina pedidos; una solicitud aprobada puede materializar un servicio; y cada servicio conserva el vínculo con su identidad técnica, capacidad reservada y período autorizado. Sobre esa estructura se relacionan operaciones, credenciales protegidas, métricas e incidentes. El diseño mantiene diferenciadas la intención académica, la autorización institucional y la instancia realmente creada.

Se adoptó un esquema relacional en PostgreSQL con migraciones versionadas para evolucionar las entidades sin perder información. Los registros de historial permiten conocer quién realizó una acción, qué decisión se tomó y cuál fue su resultado. Para datos sensibles se conserva el material cifrado y sólo se revela al perfil autorizado cuando corresponde. La elección de una base relacional responde especialmente a la necesidad de operaciones atómicas, consistencia referencial y consultas por cátedra y vigencia.

Consistencia de datos, identidades y recuperación de operaciones

El modelo de datos se diseñó para distinguir la intención del usuario de la realidad física. Un pedido representa una necesidad académica y su autorización; un servicio representa la prestación que se espera entregar; y una identidad de infraestructura vincula esa prestación con el nodo, el identificador técnico y los recursos efectivamente creados. Esta separación permite registrar un pedido aprobado cuyo despliegue todavía no comenzó, o una instancia creada cuyo servicio interno aún no está listo. Si todo se redujera a un único estado, la interfaz podría informar disponibilidad prematuramente.

El segundo problema es la concurrencia. En un sistema multiusuario, dos administradores podrían aprobar solicitudes al mismo tiempo mientras el clúster conserva sólo capacidad suficiente para una. La decisión de admisión se ejecuta dentro de una transacción de PostgreSQL, con sincronización de las evaluaciones de capacidad y registro indivisible del compromiso. Se conserva además una identidad única del recurso, de modo que un reintento de la operación no genere una segunda instancia accidental.

El proceso de ejecución trabaja de manera desacoplada de la petición web. Cuando se aprueba un pedido, la API registra una operación pendiente y responde al usuario; un worker toma la tarea y la ejecuta. Las operaciones se reclaman con bloqueo transaccional para impedir que dos

procesos trabajen sobre el mismo registro. Este diseño mantiene disponible el portal cuando una creación tarda varios minutos, y hace posible reanudar el trabajo después de una interrupción sin depender de que el navegador siga abierto.

El seguimiento de una tarea no puede apoyarse sólo en la respuesta inicial de Proxmox. Una llamada de creación o apagado puede devolver un identificador de trabajo mientras la operación continúa en el nodo. Por eso se consulta el estado de la tarea y, cuando corresponde, el estado observado del contenedor. La operación se confirma cuando existe evidencia suficiente del resultado, no cuando simplemente se envió una orden. El mismo criterio se aplica al detener y eliminar: liberar memoria o almacenamiento antes de verificar la operación produciría saldos falsos.

Una situación especialmente delicada aparece cuando se corta la conexión después de emitir una instrucción. El software puede desconocer si Proxmox llegó a ejecutarla. En ese caso se mantiene el estado como incierto, se conserva la reserva y se realiza una reconciliación con la infraestructura antes de decidir si corresponde confirmar, reparar o reintentar. El costo de una espera adicional es preferible a duplicar contenedores, borrar datos de una cátedra o comprometer recursos inexistentes.

El historial de pedidos, servicios y operaciones aporta una última propiedad: permite reconstruir qué solicitó el docente, qué autorizó administración, qué orden recibió el hipervisor y qué observó finalmente el software. Esa trazabilidad tiene valor técnico para diagnosticar incidentes y valor institucional para justificar el uso de capacidad compartida.

3. Módulos y funcionalidades del software

3.1 Identidad, usuarios y seguridad de acceso

El módulo de identidad administra el ingreso al portal y la autorización por roles. El administrador puede dar de alta a responsables de cátedra con una contraseña provisoria; el primer acceso requiere que cada usuario establezca una contraseña propia. De forma opcional se incorpora un segundo factor de autenticación mediante códigos temporales, junto con mecanismos de recuperación de cuenta.

La sesión del usuario se conserva y valida en el servidor, de modo que las operaciones sensibles puedan controlarse y revocarse. También se aplican límites ante intentos reiterados de autenticación, protección de credenciales almacenadas y restricciones para evitar que la plataforma quede sin administradores activos. Estas medidas buscan que el portal pueda ser utilizado por docentes sin sacrificar controles propios de un entorno compartido.

Para el control de acceso se distinguieron explícitamente permisos administrativos y permisos docentes. La interfaz puede simplificar la navegación según el perfil, pero la verificación decisiva se realiza en el backend al acceder a cada recurso. Esto impide que la modificación manual de una URL o la repetición de una petición permita visualizar datos de otra cátedra. Las sesiones se sostienen mediante cookies protegidas, con validaciones contra solicitudes no autorizadas y mecanismos de revocación.

El segundo factor TOTP puede habilitarse sin cambiar el proceso de alta general. El usuario recibe un código QR para registrar una aplicación autenticadora y dispone de códigos de recuperación de un solo uso. Los secretos de autenticación y las contraseñas de servicios se almacenan cifrados, mientras que los intentos fallidos se someten a límites temporales. Estas medidas son especialmente relevantes porque el portal administra accesos indirectos a recursos del clúster, aunque no exponga su consola de gestión.

3.2 Organización académica y catálogo de servicios

Este módulo representa las cátedras, sus responsables y los períodos del calendario lectivo. Administración define la vigencia de los cuatrimestres y del ciclo anual para que las solicitudes se relacionen con fechas académicas conocidas. Esto facilita el seguimiento de altas, vencimientos y renovaciones.

El catálogo presenta servicios disponibles en términos que el docente puede interpretar: tipo de servicio, descripción de uso, tamaño y requisitos básicos. Se definieron perfiles estandarizados de recursos, como Small y Medium, para simplificar decisiones y evitar la configuración arbitraria de cada instancia.

El administrador puede incorporar aplicaciones conocidas y ofertas basadas en imágenes de contenedor, incluidas imágenes propias de una cátedra. La publicación de una oferta no consume capacidad física: los recursos se asignan cuando existe un pedido aprobado. Esta decisión evita dejar instancias ociosas sólo por figurar en el catálogo.

El calendario lectivo funciona como una regla de negocio, no como un dato decorativo. Cada pedido se asocia a un intervalo autorizado y el sistema debe impedir que la instancia continúe consumiendo recursos sin una vigencia válida. Se contemplaron períodos de primer cuatrimestre, segundo cuatrimestre, anual y personalizados. La renovación extiende la autorización sobre una instancia existente, preservando en lo posible su información y accesos; no se trata de crear un servicio nuevo con cada período.

En el catálogo, los perfiles Small y Medium expresan tamaños predefinidos. Como referencia, el proyecto definió Small con 1 vCPU, 512 MB de memoria y 4 GB de disco, y Medium con 2 vCPU, 1 GB de memoria y 8 GB de disco. Mantener ofertas uniformes facilita la comparación de solicitudes y aporta previsibilidad al administrador cuando evalúa la capacidad disponible.

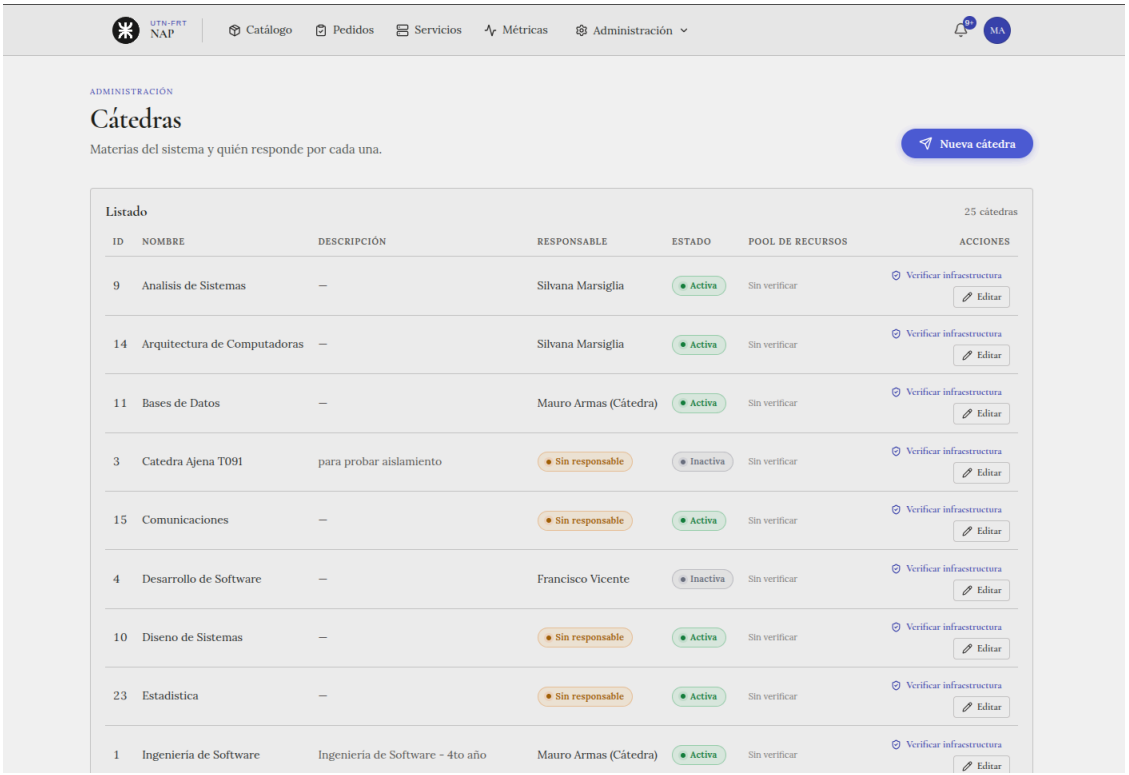


Figura 3. Administración de cátedras y responsables.

La plataforma vincula asignaturas, usuarios responsables y estado institucional de cada cátedra.

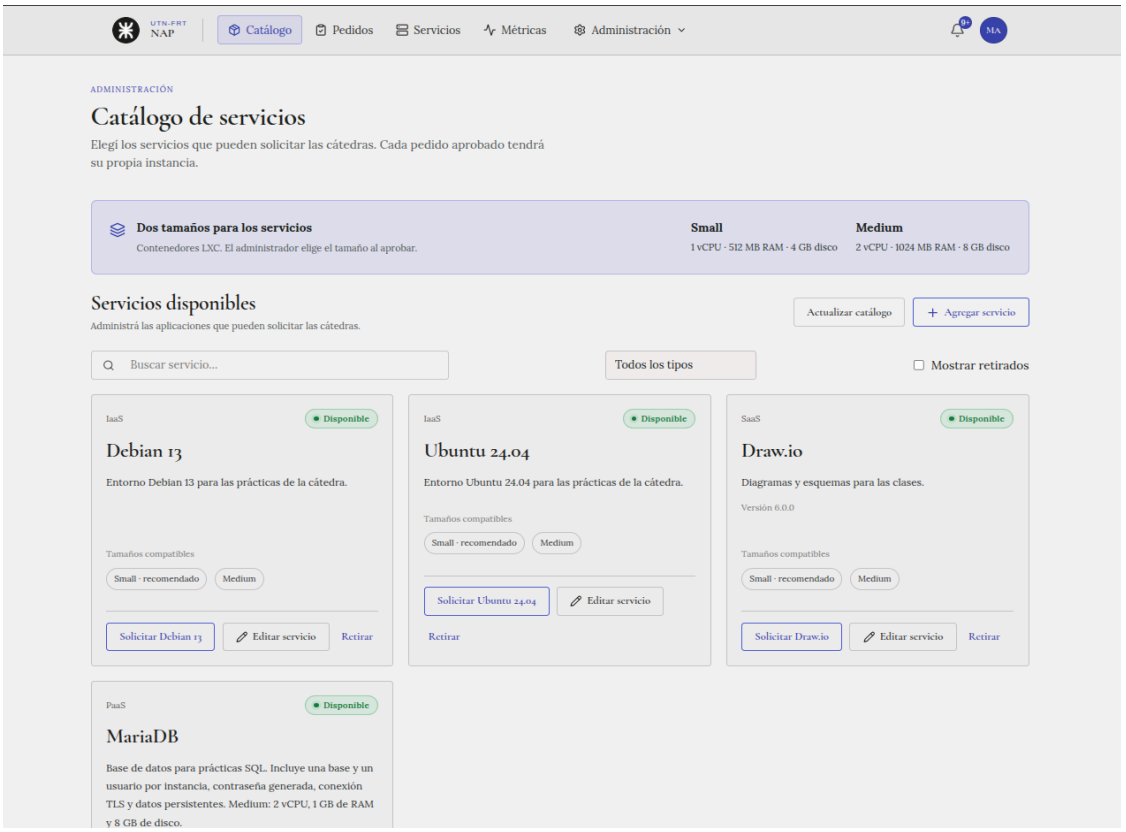


Figura 4. Catálogo de ofertas IaaS, PaaS y SaaS.

Las tarjetas muestran servicios disponibles, perfiles de recursos y opciones de solicitud y administración.

3.3 Pedidos, aprobaciones y calendario

El responsable de cátedra crea una solicitud indicando el servicio elegido, su finalidad y el período durante el que lo necesita. La administración dispone de una bandeja de pedidos con información suficiente para aprobar, rechazar o solicitar una aclaración. Las observaciones vuelven al usuario, que puede corregir y reenviar la solicitud.

La aprobación vincula dos decisiones: autorizar el uso académico y asegurar que la infraestructura tenga capacidad para cumplirlo. Si la fecha de inicio es futura, el sistema conserva la reserva y programa el despliegue para el momento correspondiente. Cuando la vigencia termina, se aplican las acciones definidas para el servicio y se notifica a los usuarios.

Se contempló la renovación de instancias existentes para no obligar al docente a reinstalar su entorno entre períodos consecutivos. También es posible deshacer una aprobación que todavía no haya originado el despliegue, siguiendo las reglas del estado del pedido.

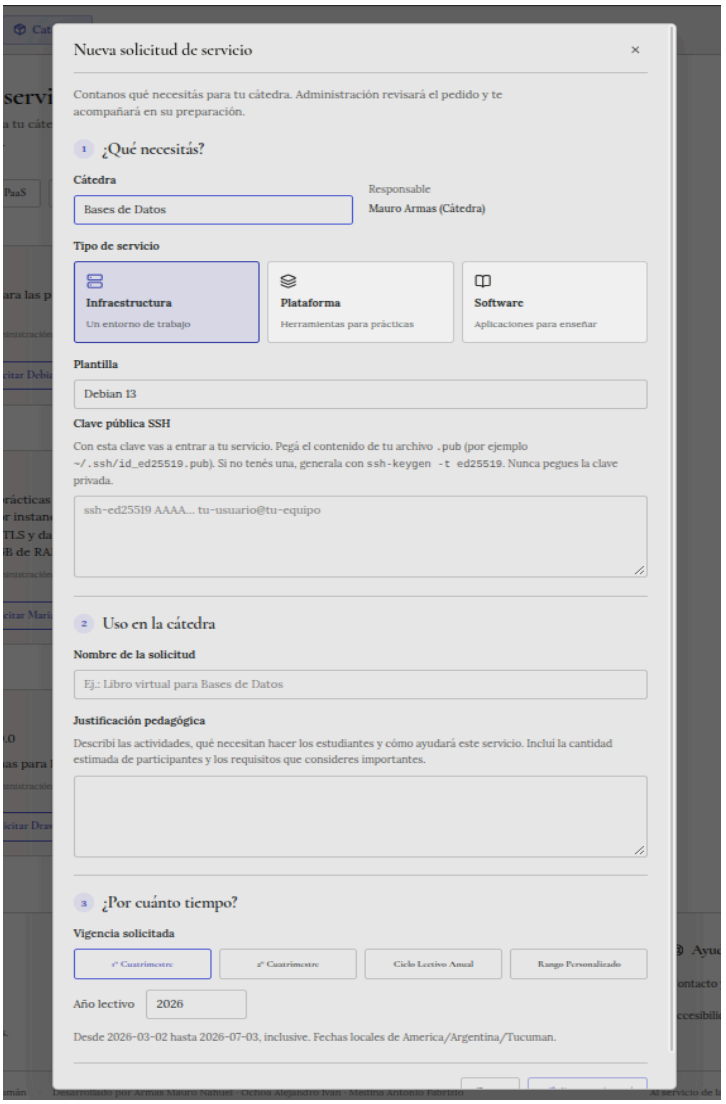


Figura 5. Formulario de solicitud académica.

El responsable selecciona servicio, justificación y vigencia; para entornos Linux puede incluir su clave pública SSH.

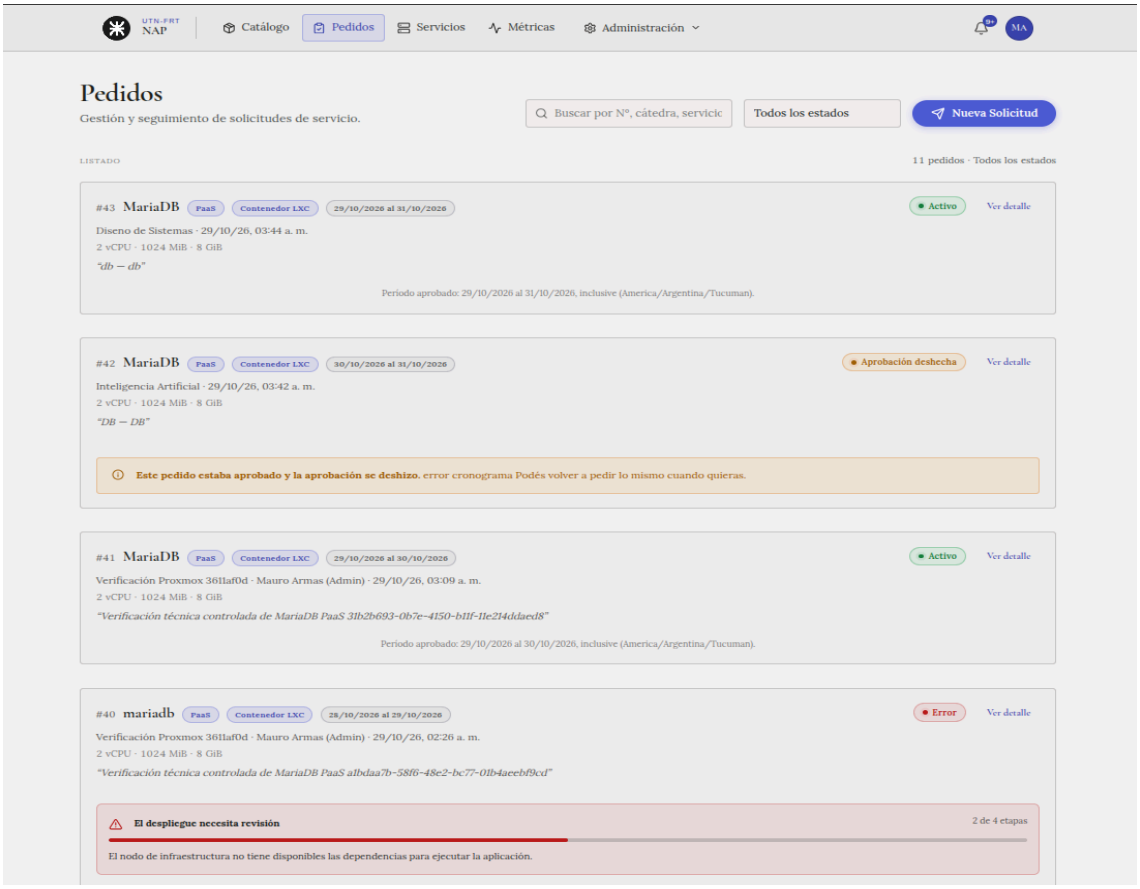


Figura 6. Seguimiento de pedidos y estados de despliegue.

La bandeja permite reconocer solicitudes activas, aprobaciones revertidas y operaciones que necesitan revisión.

3.4 Capacidad y orquestación

El módulo de capacidad consulta los recursos disponibles en el clúster y estima si un nuevo servicio puede ser atendido. Considera CPU, memoria y almacenamiento, distinguiendo los recursos que pertenecen a un nodo de aquellos que son compartidos. En lugar de asignar cuotas rígidas a cada cátedra, NAP reserva la capacidad cuando una solicitud se aprueba.

Las reservas se registran de forma transaccional para impedir que dos aprobaciones simultáneas utilicen el mismo saldo disponible. Cuando un servicio se detiene, se actualiza la disponibilidad de CPU y memoria, pero se sigue considerando el espacio ocupado por sus datos; la capacidad se libera completamente sólo después de confirmar la baja de los recursos.

El orquestador administra operaciones de creación, encendido, detención, reinicio, reparación y eliminación. Estas tareas se ejecutan de forma asíncrona y comunican estados al portal. Ante una pérdida de comunicación o un error, el sistema comprueba el estado de la infraestructura antes de repetir o dar por terminada una acción, evitando duplicaciones y liberaciones incorrectas.

La reserva de capacidad se administra como un registro contable: cada aprobación retiene CPU, memoria y espacio de disco en el destino elegido. Para calcular la factibilidad se consideran los consumos que ya existen fuera de NAP y las reservas activas del propio producto. En el caso de almacenamiento compartido, el mismo espacio no puede sumarse varias veces por estar montado en nodos diferentes. Esta precaución evita que la interfaz presente una disponibilidad artificialmente elevada.

Hay una distinción importante entre detener un servicio y eliminarlo. Cuando una instancia se detiene de forma comprobada, CPU y memoria pueden quedar disponibles para nuevas cargas, pero el disco continúa ocupado porque conserva los datos del docente. La liberación total exige

verificar la eliminación efectiva del entorno. Si una operación pierde conectividad y su resultado no está confirmado, la reserva se conserva hasta completar la reconciliación; asumir una liberación anticipada podría conducir a sobreasignación.

El mecanismo de operaciones permite trabajar con varios pedidos sin bloquear la atención web. Para cada acción se conserva un identificador, un estado y un registro de su ejecución. La aplicación evita iniciar dos operaciones incompatibles sobre un mismo servicio y admite reintentos controlados. El objetivo de estas medidas no es complejizar la experiencia, sino sostener una visión consistente cuando aparecen reinicios, latencias o respuestas parciales de infraestructura.

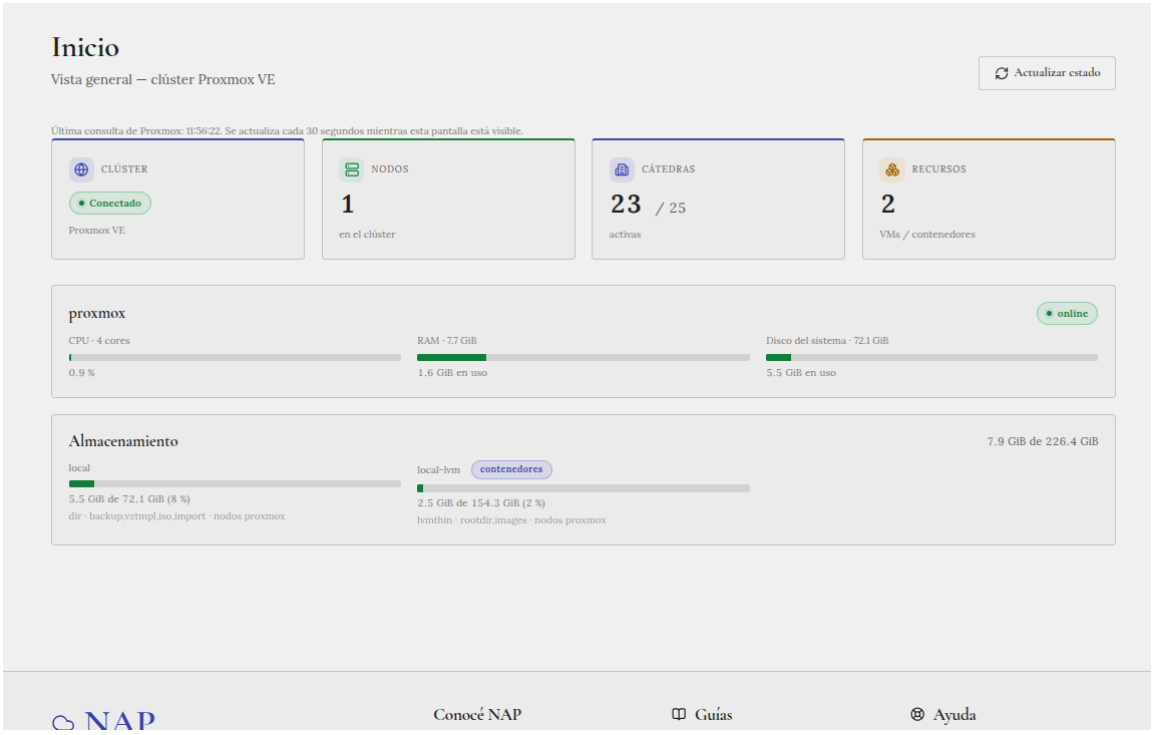


Figura 7. Vista general del clúster y de su capacidad.

El panel presenta conectividad, cantidad de nodos, recursos y utilización de CPU, memoria y almacenamiento.

Ejemplo de decisión de admisión y reserva de capacidad

El flujo de capacidad puede ilustrarse con dos solicitudes que requieren un mismo tipo de entorno. Ambos docentes seleccionan una oferta del catálogo, pero la existencia de la oferta no significa que haya dos instancias reservadas. Los pedidos se almacenan y esperan evaluación. Cuando administración aprueba el primero, el sistema consulta los nodos elegibles, descuenta la utilización que ya existe fuera de NAP y los compromisos activos, comprueba el almacenamiento autorizado y registra una reserva asociada a la solicitud y a la identidad del futuro recurso.

Si otro administrador aprueba el segundo pedido en ese mismo instante, la decisión debe utilizar el saldo actualizado, no la fotografía anterior. La sincronización transaccional evita que ambas operaciones se basen en una disponibilidad idéntica y se comprometa capacidad dos veces. El segundo pedido podrá aprobarse si existe un destino viable; de lo contrario deberá permanecer sin autorización de despliegue hasta que se liberen recursos o cambie la situación. La interfaz no debe sustituir este control por una simple advertencia.

El cálculo también cambia según el tipo de recurso. La CPU y la memoria se evalúan por nodo de ejecución, mientras que el espacio de TrueNAS montado por varios nodos se contabiliza como un recurso compartido. Sumar el espacio libre de cada montaje como si fueran discos independientes

multiplicaría artificialmente la capacidad. El orquestador debe conocer esa diferencia y preservar el compromiso de disco aunque el servicio se encuentre detenido.

Después de aprobar, el pedido puede esperar una fecha futura de inicio. La reserva continúa vigente, pero el despliegue no se ejecuta hasta el momento autorizado. Cuando llega la fecha, el worker crea la instancia, observa su estado y realiza las comprobaciones propias de la modalidad solicitada. Ante un error de aprovisionamiento, el compromiso no se elimina de forma automática: primero se determina si quedó un contenedor o un volumen parcialmente creado. Esta política permite que la reparación sea controlada y que el panel administrativo refleje recursos realmente disponibles.

3.5 Aprovisionamiento y acceso a servicios

La Nube Académica Práctica ofrece tres modalidades de consumo. Para IaaS se aprovisionan entornos Linux mediante contenedores LXC; el docente puede usar la consola y, cuando corresponde, conectarse por SSH con una clave pública. Para PaaS se dispone de una instancia de MariaDB con base de datos y usuario propios, lista para utilizarse desde herramientas o proyectos externos mediante conexión cifrada. Para SaaS se publican aplicaciones web, como Draw.io, accesibles desde el portal sin requerir administrar su instalación.

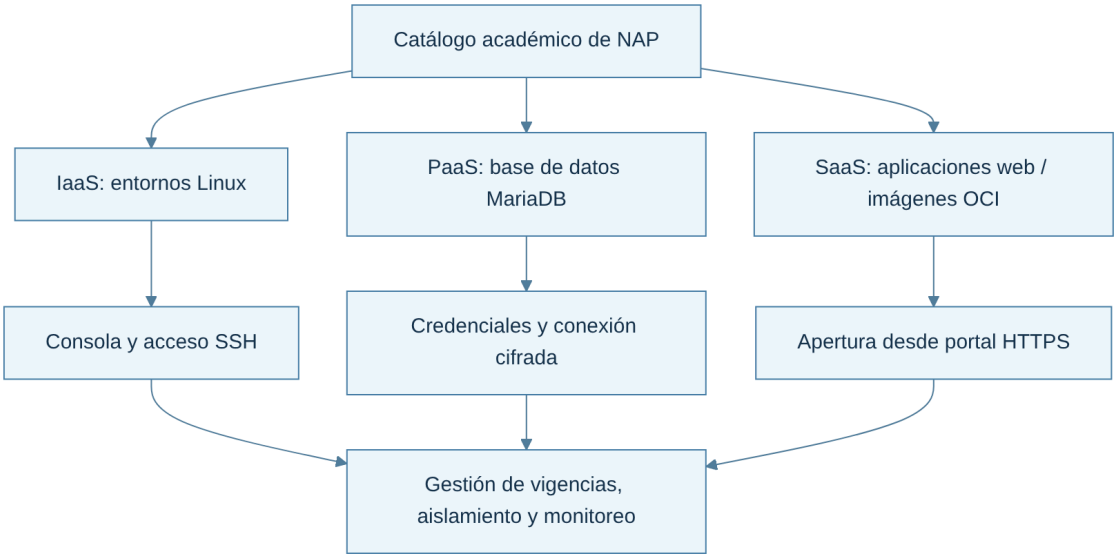


Figura 8. Modalidades de servicio ofrecidas por NAP y su forma habitual de acceso.

El catálogo unifica la experiencia de solicitud y administración, mientras que cada modalidad conserva un mecanismo de acceso apropiado a su uso académico.

En servicios de aplicaciones, la solución utiliza imágenes OCI y un motor de contenedores Docker dentro de entornos Linux aislados. Esto permite incorporar nuevas herramientas a partir de configuraciones estandarizadas, sin desarrollar un mecanismo de instalación distinto para cada software. Se incorporan verificaciones para declarar un servicio disponible sólo después de comprobar que responde correctamente.

El portal centraliza la entrega de información de conexión. En Linux muestra instrucciones para la terminal y permite gestionar la clave pública; en MariaDB presenta las credenciales específicas de la instancia; en aplicaciones web ofrece una acción para abrirlas mediante un acceso HTTPS controlado. Estas alternativas reducen pasos manuales y errores frecuentes en laboratorios docentes.

La modalidad IaaS se orienta a materias que necesitan control del entorno Linux. Se aprovisionan contenedores LXC no privilegiados con Ubuntu o Debian y se habilita acceso por consola; para uso remoto se emplean claves públicas SSH en lugar de contraseñas. Se verificó además la rotación de claves y la consulta de la dirección IP efectiva del contenedor, de modo que el portal presente instrucciones de conexión que correspondan al entorno real.

En PaaS, el usuario recibe una instancia aislada de MariaDB 11.4 con una base y un usuario propios. La conexión requiere TLS y no se entregan credenciales administrativas globales de la base de datos. En SaaS, el portal habilita aplicaciones web como Draw.io mediante un acceso HTTPS controlado y ligado a la sesión. En ambos casos se comprueba la salud del servicio antes de indicarlo como utilizable, ya que la creación del contenedor no garantiza por sí sola la disponibilidad de la aplicación.

Para ofrecer otras aplicaciones, se eligió un motor común basado en imágenes OCI y Docker dentro de un contenedor Linux. Esta arquitectura aprovecha catálogos existentes y permite publicar imágenes propias, con una política homogénea de ejecución y aislamiento. Evita depender de un procedimiento artesanal diferente para cada herramienta y facilita mantener versiones concretas de las aplicaciones desplegadas.

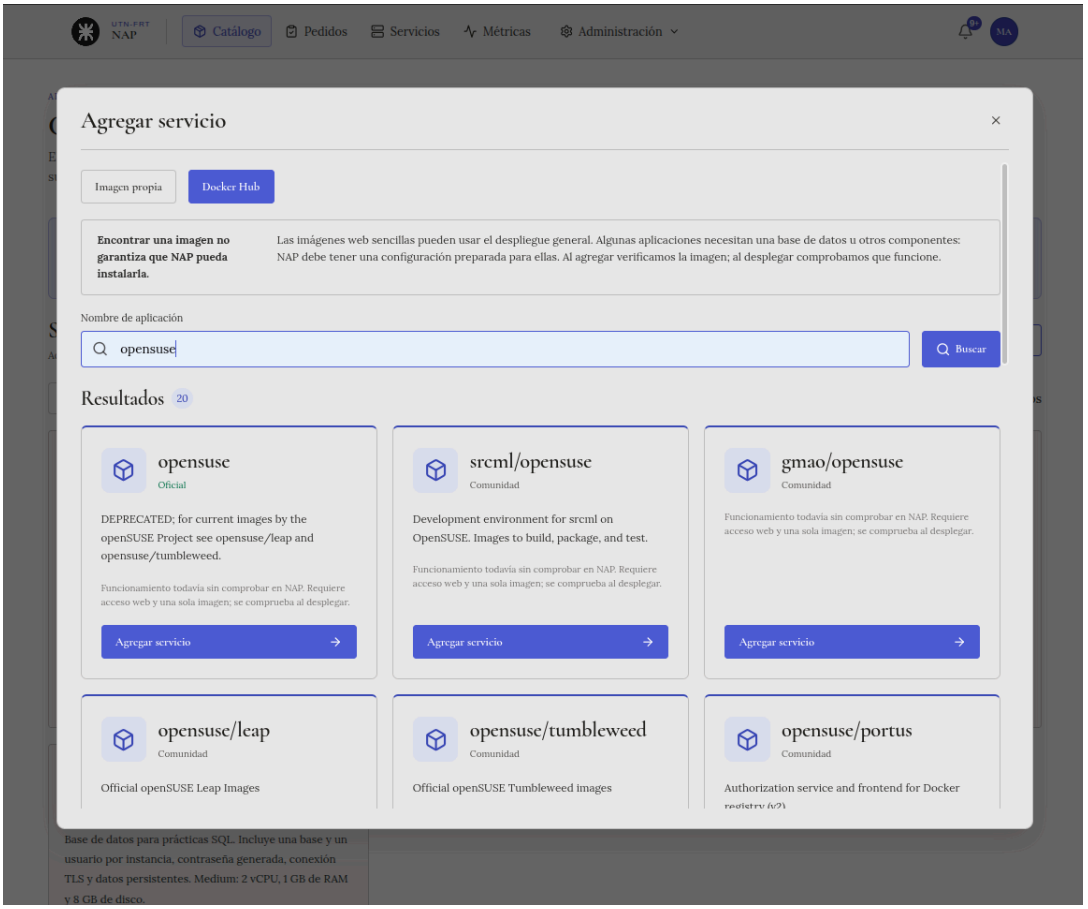


Figura 9. Incorporación de aplicaciones mediante Docker Hub.

El catálogo administrativo permite buscar imágenes externas y preparar nuevas ofertas de servicio.

3.6 Aislamiento, supervisión y notificaciones

El producto separa las cátedras tanto en el nivel de permisos del portal como en los servicios desplegados. Una cátedra no puede consultar pedidos, credenciales ni métricas ajenas. Para las aplicaciones alojadas en el clúster se aplican reglas de red destinadas a impedir la comunicación con otros entornos académicos y con segmentos sensibles de administración y almacenamiento.

La seguridad no depende únicamente de que un nodo permanezca encendido: se diseñó un mecanismo de restauración de las reglas de aislamiento después de un reinicio. Además, el orquestador verifica el estado real de las instancias y permite reparar despliegues interrumpidos conservando la identidad del servicio cuando resulta seguro hacerlo.

El módulo de observabilidad recopila periódicamente indicadores de CPU, memoria, disco y tráfico de red. El docente obtiene gráficos sobre sus servicios, mientras que el administrador dispone de una vista general del estado del clúster. El historial combina muestras de detalle con resúmenes diarios, permitiendo analizar el uso durante períodos académicos sin conservar indefinidamente todos los registros de alta frecuencia.

NAP genera alertas ante interrupciones o consumo sostenido de recursos y detecta entornos potencialmente inactivos. También informa mediante notificaciones internas y correo sobre cambios relevantes: pedido recibido o resuelto, servicio disponible, vencimiento próximo e incidentes de operación.

El aislamiento combina controles de aplicación y de red. El primero restringe las consultas y operaciones por cátedra; el segundo limita el tráfico entre instancias y protege las redes sensibles de administración y almacenamiento. Sobre los nodos de Proxmox, estas políticas se implementan mediante filtrado de tráfico y se restauran durante el arranque, antes de habilitar el uso de servicios. El propósito es que una actividad de laboratorio no pueda interferir accidentalmente con otra.

Las métricas se recogen a intervalos de un minuto y consideran utilización de CPU, memoria, disco y tráfico de red. Para este último indicador, el sistema calcula variaciones entre mediciones, en lugar de interpretar el total acumulado como consumo instantáneo. La consolidación diaria permite consultar períodos extensos de actividad manteniendo control sobre el volumen histórico de datos.

En la capa de supervisión se reconocen incidentes por caída o saturación prolongada y se identifica la inactividad de servicios durante un período definido. Los avisos se entregan en el portal y por correo electrónico según la naturaleza del evento. Esta observabilidad cumple una doble función: ayudar al usuario a conocer la situación de su entorno y proporcionar a administración datos para decidir futuras asignaciones.

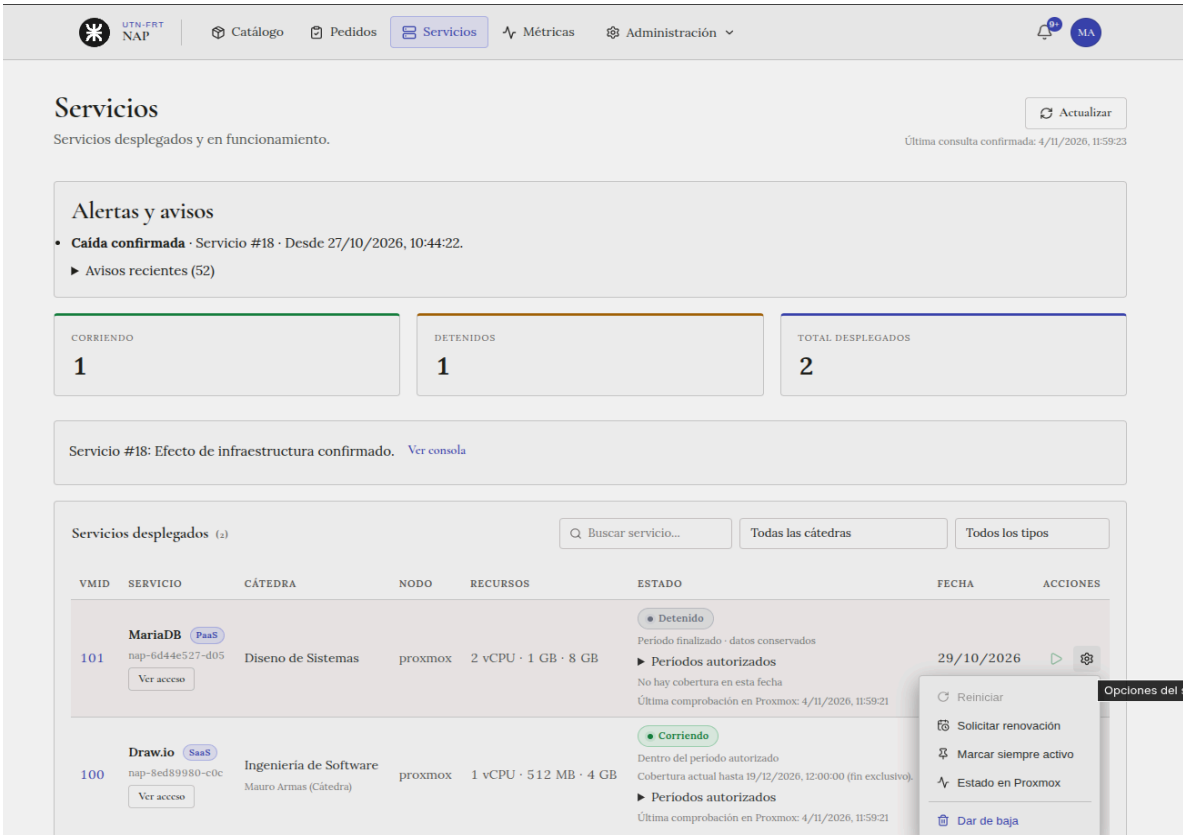


Figura 10. Panel de servicios y controles disponibles.

La vista reúne instancias desplegadas, estados, alertas y acciones de operación o renovación.

3.7 Experiencia de usuario y administración

La interfaz se organizó para que cada perfil vea únicamente las acciones y los datos necesarios. El responsable de cátedra cuenta con un panel de pedidos y servicios, indicadores de avance del despliegue, controles de operación y accesos listos para utilizar. El administrador ve adicionalmente el estado de capacidad, el catálogo, las aprobaciones, las cátedras, los usuarios y las tareas de supervisión.

Se priorizó mostrar los estados de manera comprensible y registrar información útil cuando una acción falla. La pantalla de seguimiento distingue etapas como aprobación, creación del entorno, configuración del servicio y verificación de acceso. Así se evita que una operación de varios minutos sea percibida como una pantalla detenida.

Para facilitar la transferencia del producto, el sistema se acompañó con mecanismos de despliegue reproducible, procedimientos de configuración y pautas de recuperación. En caso de restaurar datos del portal, se exige comprobar la correspondencia con el estado actual del clúster antes de reanudar acciones automáticas, reduciendo el riesgo de actuar sobre información desactualizada.

La experiencia de usuario fue diseñada para ocultar complejidad sin ocultar información importante. Las tareas que toman tiempo muestran etapas de avance; los mensajes de error ofrecen contexto para comprender qué ocurrió; y los accesos a IaaS, PaaS y SaaS se presentan como instrucciones y acciones específicas de cada modalidad. En vez de mostrar identificadores internos como eje de navegación, el portal prioriza los nombres de cátedra, servicio y período académico.

El administrador cuenta con paneles para usuarios, cátedras, catálogo, inventario, solicitudes pendientes, utilización del clúster y estado de servicios. Esta visión transversal permite detectar

solicitudes inviables antes de aprobarlas y reconocer recursos que siguen asignados a pesar de estar detenidos. Para el responsable de cátedra, la misma información se reduce deliberadamente al subconjunto propio de sus asignaturas.

4. Decisiones técnicas y criterios de calidad

4.1 Tecnologías elegidas

El frontend se desarrolló con React y Vite; la API con Python y FastAPI; y la persistencia con PostgreSQL. La ejecución de operaciones se separó en un proceso de trabajo asíncrono. Para el aprovisionamiento se integró el portal con Proxmox VE y se utilizaron tecnologías de contenedores Linux y Docker. La publicación y el acceso a los servicios emplean mecanismos estándar como HTTPS y SSH.

La combinación se eligió por su adecuación al contexto: interfaces web conocidas para los usuarios, un backend mantenible, una base de datos relacional capaz de preservar consistencia, y una infraestructura de contenedores que aprovecha los recursos existentes sin introducir una plataforma de nube adicional de gran consumo.

Se consideró la alternativa de utilizar directamente la consola del hipervisor o adoptar gestores generales de nube. Sin embargo, ambas opciones exigían al docente una complejidad innecesaria o agregaban componentes poco alineados con el proceso académico. Un desarrollo especializado permitió traducir la solicitud por cátedra y período lectivo a operaciones automatizadas de menor alcance, pero más apropiadas al problema.

4.2 Seguridad, recuperación y mantenibilidad

Las decisiones de seguridad se apoyaron en permisos por rol, protección de sesiones, cifrado de secretos, acceso limitado a la infraestructura y separación de tráfico entre servicios. Para las bases de datos se exige comunicación cifrada; para Linux se admite acceso mediante clave pública; para aplicaciones web se utiliza un mecanismo de acceso vinculado a la sesión del portal.

La consistencia operativa fue otro criterio relevante. El software no considera completa una operación hasta comprobar el resultado, y conserva reservas cuando el estado de un recurso es incierto. El registro de eventos y las posibilidades de reparación ayudan a explicar fallos y retomar el proceso sin recrear indiscriminadamente los entornos.

Por último, se cuidó la separación modular para que la incorporación de nuevas aplicaciones, el ajuste de políticas académicas o una evolución de infraestructura no obliguen a reescribir toda la solución. Estas decisiones contribuyen a que el producto pueda ser mantenido por personal técnico posterior a la PPS.

4.3 Evaluación de alternativas y compromisos

La elección de un desarrollo a medida se comparó con administrar los recursos desde la interfaz original de Proxmox o incorporar una plataforma de nube de propósito general. La primera opción expone demasiadas decisiones técnicas al docente; la segunda incorpora capacidades y costos operativos innecesarios para el volumen y los procesos académicos previstos. NAP conserva el hipervisor como componente especializado y agrega una capa institucional ligera que gobierna el consumo de recursos.

Se prefirió una cola de trabajo respaldada por la misma base de datos transaccional, en lugar de incorporar un servidor de mensajería independiente. Esta decisión reduce componentes que administrar en un entorno de memoria limitada, aunque exige especial cuidado en el control de concurrencia y recuperación de operaciones. También se eligieron contenedores LXC como unidad de cómputo para aprovechar su menor sobrecarga frente a máquinas virtuales completas; para las aplicaciones se añadió una segunda capa OCI controlada dentro de cada entorno.

En seguridad, se separó la autenticación del portal del acceso a los servicios. El portal decide quién puede solicitar y administrar; SSH, TLS y el gateway web resuelven cómo se ingresa al recurso asignado. Esa división permite aplicar políticas diferentes según el tipo de servicio sin confundir las credenciales de la plataforma con las de cada entorno académico.

Criterios de integración y límites del plano de control

Una decisión central fue evitar que el portal actuara como una consola alternativa de Proxmox. La interfaz académica no necesita exponer identificadores de máquinas, bridges, parámetros de disco o rutas de almacenamiento para cada operación cotidiana. En cambio, el software recibe una intención de alto nivel —servicio, tamaño, cátedra y período— y la transforma en una secuencia de comprobaciones y acciones autorizadas. Este criterio reduce errores de operación y hace posible que la administración mantenga una política homogénea aun cuando cambie el equipamiento físico.

El acceso a Proxmox se resolvió mediante una cuenta técnica y un token de API con permisos acotados a las tareas requeridas. Se evitó reutilizar credenciales personales de los administradores de infraestructura. El backend consulta inventario, nodos y capacidad, mientras que el worker ejecuta acciones de creación y ciclo de vida; ambos deben validar la identidad y los límites del recurso antes de modificarlo. El token se conserva como secreto de configuración del servicio y no forma parte de las respuestas al navegador. El transporte utiliza HTTPS y la verificación de certificados debe completarse con la autoridad de confianza del entorno institucional.

Para operaciones que requieren actuar dentro de un contenedor, el proyecto utiliza un ejecutor de nodo separado del acceso REST al hipervisor. Su propósito es aceptar acciones previstas y parámetros validados, no abrir una sesión de shell arbitraria desde el portal. Así se distinguen tres ámbitos de responsabilidad: la API del producto verifica la autorización académica; Proxmox administra los recursos de virtualización; y el ejecutor realiza la personalización acotada que la API de infraestructura no proporciona directamente. El fallo de una de estas capas no debería otorgar permisos irrestrictos sobre las otras.

También se consideró la portabilidad de la configuración. Los nombres de nodos, los almacenamientos disponibles y las rutas de red no deben codificarse como supuestos universales del producto. La conexión con el clúster físico, de cuatro nodos tras descartarse equipos defectuosos, confirmó la necesidad de descubrir el inventario real y seleccionar destinos elegibles. En los laboratorios se utilizaron otras identidades de nodo y redes; las mismas reglas de negocio debían seguir funcionando sin cambiar el flujo del docente.

El almacenamiento compartido exige una precaución adicional. La presencia de un recurso NFS montado en varios nodos facilita ejecutar instancias en distintos servidores, pero no convierte ese almacenamiento en cuatro capacidades independientes. El orquestador debe reconocer su ámbito compartido, descontar una sola vez los compromisos de disco y mantenerlos mientras los datos existan. De manera semejante, una caída de conectividad con el storage no autoriza a asumir que un volumen desapareció.

Estas decisiones expresan el compromiso entre simplicidad de uso y rigor operativo. La complejidad de permisos, recursos y recuperación permanece en el software y en sus interfaces controladas; la experiencia docente sólo expone la información necesaria para solicitar, usar y renovar servicios.

5. Validación del producto y resultados

5.1 Estrategia de pruebas

La validación combinó pruebas automatizadas de reglas del negocio y componentes, pruebas de integración con la base de datos, recorridos completos en navegador, ensayos de rendimiento y experimentación sobre hipervisores reales. El objetivo fue comprobar tanto la experiencia de los dos perfiles de usuario como las condiciones que suelen provocar fallos en un orquestador: aprobaciones concurrentes, comunicaciones interrumpidas, errores de despliegue, aislamiento entre cátedras y cambios de estado del clúster.

De acuerdo con los resultados consignados en los registros del proyecto, se ejecutaron más de 900 pruebas automatizadas, entre backend, frontend, operaciones de infraestructura y recorridos de usuario. Las suites documentadas finalizaron sin fallos en sus ejecuciones de cierre. Esta evidencia respalda el comportamiento ensayado, aunque no sustituye las pruebas de aceptación en el entorno productivo definitivo.

Las pruebas se organizaron por niveles para detectar defectos en etapas distintas: pruebas unitarias para reglas de negocio, integraciones para persistencia y concurrencia, recorridos completos desde el navegador y validación sobre hipervisores reales. Se ensayaron escenarios como aprobaciones simultáneas, intentos de acceso cruzado entre cátedras, vencimientos, cancelaciones, reintentos de operaciones y credenciales incorrectas. En estos casos, el criterio de aceptación no fue únicamente que la interfaz respondiera, sino que el estado persistido se correspondiera con las reglas definidas.

La verificación de concurrencia utilizó PostgreSQL real para comprobar el comportamiento de los bloqueos transaccionales y el reclamo de operaciones. Esta distinción es relevante: una prueba aislada de lógica puede aprobar aun cuando dos solicitudes concurrentes produzcan un resultado incorrecto sobre una base de datos. La combinación de pruebas en memoria y pruebas con el motor relacional permitió cubrir ambos tipos de riesgo.

5.2 Rendimiento bajo carga

Se evaluó el portal con una base de prueba de 100 servicios y 12,96 millones de registros históricos de métricas, manteniendo 10 sesiones autenticadas concurrentes. El criterio de aceptación fue que el percentil 95 de los tiempos de respuesta de las vistas evaluadas permaneciera dentro de 3 segundos.

Vista	p_{95} medido	Criterio	Resultado
Bandeja de pedidos	1,57 s	≤ 3 s	Cumplido
Servicios	2,75 s	≤ 3 s	Cumplido
Métricas	2,47 s	≤ 3 s	Cumplido

Tabla 1. Resultados medidos del ensayo de rendimiento en las vistas principales del portal.

Las tres vistas cumplieron el criterio en el escenario ensayado. Se trata de una medición bajo condiciones reproducibles de laboratorio, no de una garantía universal de rendimiento en cualquier despliegue.

5.3 Validación sobre infraestructura real

Se probó el ciclo de vida de entornos Linux desde la solicitud y aprobación hasta el inicio, detención y baja; el acceso SSH mediante clave pública y su rotación; el despliegue de aplicaciones web con HTTPS; y el aprovisionamiento de MariaDB con credenciales particulares y conexión cifrada. También se verificó que los servicios mantuvieran sus datos tras detenerse y volver a iniciarse.

En un laboratorio de dos nodos Proxmox con almacenamiento TrueNAS compartido se desplegaron servicios para cátedras distintas y se comprobaron restricciones de acceso, separación de datos y reglas de aislamiento de red. Se ensayaron además errores deliberados de inicialización para validar la operación de reparación, y un reinicio de nodo para comprobar la recuperación del aislamiento y el seguimiento del estado de los servicios.

Estas pruebas aportaron evidencia de funcionamiento integrado más allá de la simulación. Particularmente, permitieron detectar situaciones que no surgían en entornos de desarrollo: tiempos de inicialización mayores sobre almacenamiento compartido, diferencias en la asignación real de direcciones de red y necesidad de reconstruir políticas de seguridad tras reinicios.

En la validación física se ensayó tanto el recorrido positivo como el rechazo de condiciones inseguras. En Linux se comprobó el acceso por clave pública y el bloqueo de autenticación por contraseña o del ingreso remoto administrativo. En MariaDB se verificó una conexión cifrada, la persistencia de datos tras reinicios y el rechazo de conexiones sin TLS o con credenciales inválidas. Para aplicaciones web se confirmó la disponibilidad por HTTPS y el funcionamiento del acceso desde el portal.

Además, el trabajo de laboratorio de dos nodos permitió observar el comportamiento del aprovisionamiento sobre almacenamiento compartido y la persistencia del aislamiento entre cátedras tras reiniciar un nodo. Se provocaron fallos controlados de inicialización para evaluar la recuperación de un servicio sin duplicarlo y sin descartar sus datos. Estas pruebas complementaron la regresión automatizada porque involucran componentes que sólo manifiestan ciertas condiciones en infraestructura real.

Conexión con el clúster físico de la facultad y alcance de la integración

Una parte relevante de la PPS consistió en trasladar el software desde entornos de desarrollo y laboratorios controlados hacia la infraestructura física de la UTN-FRT. Este paso requirió coordinar el contrato de integración con las otras dos responsabilidades del proyecto: el clúster Proxmox VE y su red, preparados en el trabajo de infraestructura, y el almacenamiento TrueNAS compartido, preparado en el trabajo de almacenamiento. La intervención propia de esta práctica fue adaptar el portal y el orquestador para consumir esas capacidades mediante interfaces programáticas, sin convertir al usuario docente en administrador del hipervisor.

El plan original contemplaba cinco nodos de virtualización. Las fallas de hardware documentadas durante la implementación llevaron a conformar el entorno físico definitivo con cuatro nodos operativos, denominados nap1, nap2, nap3 y nap4. La reducción no modificó el objetivo del middleware, pero sí obligó a consultar la capacidad efectiva del clúster en lugar de suponer una cantidad fija de servidores. La red de procesamiento y administración se diferenció de la red dedicada al almacenamiento; esta última permitió que los nodos accedieran al servidor TrueNAS mediante NFS. Para el software, la consecuencia práctica es que el inventario debe reconocer recursos compartidos y no sumar cuatro veces el mismo espacio libre.

El punto de entrada al clúster fue la API REST de Proxmox VE, expuesta mediante HTTPS en el puerto 8006. En la infraestructura se preparó una identidad específica para el orquestador, un rol con privilegios acotados y un token de API. El backend utiliza esas credenciales de servicio para consultar los nodos, su utilización y los recursos existentes; el docente nunca recibe el token ni necesita conocer las direcciones administrativas. La integración diferencia las consultas de inventario de las acciones que modifican recursos: el portal presenta información y registra pedidos, mientras que el proceso de ejecución envía las operaciones autorizadas y sigue su resultado.

En la prueba de conexión se utilizaron, entre otras, las operaciones GET /cluster/resources para obtener el inventario global, GET /nodes/{node}/status para conocer la situación del nodo elegido y GET /nodes/{node}/tasks/{upid}/status para seguir tareas que Proxmox procesa de manera asíncrona. La evidencia del equipo de infraestructura registra además la clonación y puesta en marcha de una máquina virtual desde el software mediante las rutas correspondientes de la API. Esa prueba comprobó el recorrido de extremo a extremo entre la capa de gestión y el clúster físico, con el disco de la instancia alojado en el almacenamiento TrueNAS. No debe confundirse con la aceptación completa de todos los servicios IaaS, PaaS y SaaS sobre los cuatro nodos definitivos.

En la implementación del producto, la gestión de servicios Linux y aplicaciones se apoya principalmente en contenedores LXC. Para esas operaciones el orquestador consulta los nodos y los almacenamientos autorizados, reserva capacidad, asigna una identidad técnica al recurso, solicita la creación o modificación en Proxmox y espera la finalización de la tarea. Cuando se necesita personalizar el interior de un contenedor o aplicar políticas de aislamiento que exceden la API de gestión, se utiliza un canal separado hacia un ejecutor restringido en el nodo. Esta separación es deliberada: un token de API no debe equivaler a una consola general con permisos ilimitados.

La integración física confirmó la viabilidad del enlace entre las capas y el aprovisionamiento básico. Las pruebas más profundas de seguridad, reparación, acceso SSH, MariaDB con TLS, aislamiento entre cátedras y recuperación después de reinicios se realizaron en laboratorios Proxmox reales, incluido un entorno de dos nodos con TrueNAS compartido. Al cierre de la PPS todavía faltaba repetir ese protocolo avanzado completo sobre nap1 a nap4, terminar la configuración institucional de certificados y accesos, y realizar la aceptación con docentes de la red universitaria. Presentar por separado estos niveles de evidencia permite describir un resultado concreto sin atribuir al clúster definitivo validaciones que aún estaban pendientes.

5.4 Alcance de las evidencias y pruebas pendientes

El software también se conectó con los servidores físicos del entorno académico para verificar operaciones básicas de aprovisionamiento. No obstante, al cierre de la PPS quedó pendiente repetir el protocolo avanzado completo sobre los cuatro nodos definitivos de la facultad, finalizar los certificados y configuraciones institucionales y ejecutar pruebas de aceptación con docentes reales en la red universitaria. Estas actividades corresponden a la puesta en producción y no deben presentarse como realizadas.

Por lo tanto, el estado del producto se describe como desarrollado y validado funcionalmente en laboratorio, con integración básica comprobada en el entorno definitivo y con acciones finales de despliegue institucional pendientes.

La evidencia debe interpretarse según su alcance. Una suite automatizada aprobada demuestra que el comportamiento previsto se cumple en el escenario que representa; un ensayo de rendimiento mide un ambiente y una carga definidos; una prueba en el laboratorio multinodo valida la integración de sus componentes. Ninguna de estas evidencias permite afirmar por sí sola que la operación productiva del clúster de cuatro nodos ya fue aceptada institucionalmente. Por ello, el informe separa deliberadamente los resultados obtenidos de los hitos pendientes de puesta en servicio.

Lectura de las evidencias y criterios de aceptación pendientes

Las evidencias del proyecto deben interpretarse en tres niveles. El primero es la validación del software: pruebas de componentes, API, persistencia, concurrencia y recorridos de navegador. Permiten concluir que las reglas y flujos previstos se comportaron correctamente en los entornos ensayados. El segundo nivel es la validación de integración en laboratorio real, donde intervienen Proxmox VE, contenedores, almacenamiento compartido y redes. El tercero es la aceptación institucional sobre los cuatro nodos definitivos y con usuarios reales. Los dos primeros cuentan con resultados documentados; el tercero conserva actividades pendientes.

Como evidencia funcional representativa se registró el recorrido completo de un entorno Linux: solicitud de una oferta, aprobación, reserva de recursos, creación del contenedor, inicio, detención y baja. La comprobación relevante no es únicamente la aparición del contenedor, sino la correspondencia entre su estado observado y el inventario del portal, incluido el comportamiento distinto de la reserva de CPU/memoria y del espacio persistente. Esta secuencia respalda el diseño del ciclo de vida, aunque no constituye por sí sola una prueba de alta disponibilidad del clúster.

Para PaaS se ensayó una instancia MariaDB con credenciales propias y conexión TLS. La aceptación incluyó una operación de consulta, escritura y posterior lectura de datos después de detener e iniciar el servicio, así como rechazos de contraseñas incorrectas, conexiones sin cifrado y accesos administrativos no permitidos. Para SaaS se verificó el acceso HTTPS y la comprobación de salud de aplicaciones desplegadas. Estos casos muestran por qué la condición de servicio listo debe basarse en pruebas específicas y no únicamente en el estado de ejecución del contenedor.

En el laboratorio multinodo se desplegaron servicios de cátedras diferentes sobre dos nodos con TrueNAS compartido. Las verificaciones abarcaron la imposibilidad de consultar servicios ajenos desde el portal, el rechazo de credenciales cruzadas y el bloqueo de comunicaciones hacia otras instancias y redes sensibles. También se reinició un nodo y se comprobó la restauración de las reglas de aislamiento antes de volver a operar los servicios. Son evidencias de integración más fuertes que una prueba simulada, pero se circunscriben a la configuración del laboratorio utilizado.

El informe de infraestructura del equipo registra, por su parte, una prueba de creación de máquina virtual desde el software sobre el clúster físico nap1 a nap4, utilizando la API de Proxmox y el

almacenamiento TrueNAS. Este resultado valida la conexión real entre capas. No debe confundirse con la ejecución completa de las compuertas de aplicaciones, aislamiento, certificados y recuperación en los cuatro nodos finales. La diferencia es importante porque evita atribuir a una prueba de aprovisionamiento básico garantías que todavía no fueron medidas.

Para la transferencia institucional quedan como criterios de salida la configuración final de credenciales y certificados, la ejecución del protocolo de pruebas avanzadas en cada nodo, la comprobación de políticas de red y respaldo en el entorno definitivo, y la aceptación por responsables de cátedra. También deben documentarse los resultados de migración en vivo, alta disponibilidad y carga del clúster, que el informe de infraestructura identifica como no ejecutados durante el período de la PPS. Estas tareas no desmerecen el trabajo realizado: delimitan el punto exacto desde el cual debe continuar la puesta en servicio.

6. Problemas relevantes y mejoras incorporadas

6.1 Ajustes de arquitectura y seguridad

Una revisión interna permitió detectar y corregir problemas que afectaban la confiabilidad del prototipo. Entre los más significativos se encontraban operaciones que podían bloquear la respuesta del portal, cambios de estado informados antes de la confirmación del hipervisor y un cálculo incorrecto del tráfico de red al usar contadores acumulados. Se resolvieron separando la ejecución de tareas, confirmando los resultados reales y calculando el consumo por intervalos.

También se reforzaron las verificaciones de seguridad, los controles de sesión y las dependencias de la aplicación. La importancia de estas mejoras no reside en los archivos modificados, sino en su efecto sobre el producto: menor riesgo de inconsistencias, mejor disponibilidad del portal y métricas útiles para tomar decisiones.

6.2 Simplificación del despliegue de aplicaciones

El primer enfoque contemplaba utilizar scripts externos diferentes para instalar aplicaciones. Durante la integración se evidenciaron dificultades para controlar sus cambios y repetir los resultados. Se reemplazó esa estrategia por un mecanismo común basado en imágenes OCI y contenedores Docker. Con ello, la oferta académica puede ampliarse sobre una base más uniforme, sujeta a las mismas políticas de seguridad y verificación.

La elección también simplifica la administración: incorporar una aplicación compatible deja de exigir diseñar un procedimiento de instalación específico. Se mantiene, sin embargo, una validación previa de compatibilidad y seguridad antes de habilitar una imagen como oferta utilizable.

6.3 Aprendizajes de la integración

Las pruebas reales mostraron que una aplicación puede necesitar más tiempo para iniciar cuando trabaja sobre almacenamiento compartido. Se ajustaron los criterios de espera y se mantuvo la comprobación de disponibilidad antes de entregar el servicio. También se fortaleció la reparación de despliegues parciales para evitar duplicar instancias o sobrescribir datos ya inicializados.

Por otra parte, se contempló que la dirección de red efectiva de un contenedor puede diferir de la prevista. El portal pasó a informar la dirección realmente comprobada para el acceso por SSH. Finalmente, se incorporó la restauración automática de reglas de aislamiento al reiniciar un nodo, una condición necesaria para sostener las políticas de seguridad durante la operación.

En conjunto, estos ajustes reflejan un criterio de ingeniería orientado al comportamiento observable del sistema y a la experiencia de uso, más que a la mera finalización de tareas de programación.

Un hallazgo transversal fue que la corrección funcional no basta cuando una plataforma administra recursos compartidos. Los errores en estados, reintentos o decisiones de liberación pueden generar efectos sobre más de una cátedra. Por esa razón se priorizaron los mecanismos de admisión atómica, verificación posterior de operaciones y recuperación ante situaciones inciertas. Estas mejoras incrementaron la robustez operacional sin exigir nuevas acciones al usuario final.

7. Cumplimiento del plan de trabajo (200 horas)

La práctica se desarrolló del 14 de septiembre al 3 de noviembre de 2026. Las 200 horas previstas se distribuyeron en ocho etapas, desde el relevamiento inicial hasta la documentación y entrega. Las actividades se presentan a continuación de forma sintética, manteniendo la correspondencia con el plan original.

Fase	Actividad	Horas
1	Relevamiento y requisitos	20
2	Diseño de arquitectura y datos	25
3	Backend y orquestación	45
4	Portal web y experiencia de usuario	30
5	Seguridad, observabilidad y rendimiento	25
6	Servicios IaaS, PaaS y SaaS	30
7	Pruebas de laboratorio y ajustes	15
8	Documentación, transferencia e informe	10
Total	**Práctica Profesional Supervisada**	**200**

Tabla 2. Distribución de las 200 horas de Práctica Profesional Supervisada.

Las actividades del plan se completaron dentro del período informado. La validación pendiente en los cuatro nodos físicos definitivos se consigna por separado porque depende de la preparación final del entorno operacional y no modifica la carga horaria realizada.

8. Conclusión y líneas de evolución

8.1 Conclusión

El resultado de esta Práctica Profesional Supervisada es una capa de software que transforma recursos de virtualización en servicios académicos solicitables y administrables. NAP ofrece al docente un proceso simple y al administrador un control centralizado sobre autorizaciones, capacidad, despliegue y seguimiento. Su diseño responde a un problema concreto de la institución y articula funcionalidades de ingeniería de software con un entorno de infraestructura real.

La implementación permitió integrar interfaz web, reglas de negocio, persistencia, ejecución asíncrona y servicios de infraestructura bajo criterios de modularidad, seguridad y trazabilidad. Las pruebas automatizadas y las validaciones reales en laboratorio demostraron el funcionamiento de los principales recorridos y ayudaron a corregir defectos de integración. El cierre de la práctica distingue de manera explícita esos resultados de las actividades aún necesarias para operar en los cuatro nodos definitivos.

Desde el punto de vista profesional, el valor del trabajo individual no radica sólo en haber construido una aplicación, sino en haber diseñado un producto capaz de responder a requerimientos de usuarios, asignar recursos con consistencia, manejar condiciones de error y comunicar claramente su estado.

8.2 Trabajo futuro

• Completar la instalación y aceptación del software en los cuatro nodos definitivos de la facultad, incluidos accesos, certificados y pruebas con docentes.

• Integrar el inicio de sesión con la identidad institucional y mejorar la publicación de nombres de acceso dentro de la red académica.

• Incorporar nuevas aplicaciones validadas al catálogo y ampliar los servicios útiles para diferentes cátedras.

• Extender las funciones de respaldo y recuperación bajo demanda desde el portal.

• Evaluar la reactivación automática y segura de servicios tras reinicios de los nodos, distinguiendo detenciones voluntarias de interrupciones operativas.

9. Referencias

Mell, P., y Grance, T. (2011). The NIST Definition of Cloud Computing. NIST Special Publication 800-145.

Liu, F., et al. (2011). NIST Cloud Computing Reference Architecture. NIST Special Publication 500-292.

Proxmox Server Solutions GmbH. Proxmox VE Administration Guide y documentación de la API REST. https://pve.proxmox.com/pve-docs/

Hevner, A. R., et al. (2004). Design Science in Information Systems Research. MIS Quarterly, 28(1), 75–105.

Peffers, K., et al. (2007). A Design Science Research Methodology for Information Systems Research. Journal of Management Information Systems, 24(3), 45–77.

Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. Linux Journal, 239.

FastAPI. Documentación oficial. https://fastapi.tiangolo.com/

The PostgreSQL Global Development Group. PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/

Open Container Initiative. Image y Runtime Specifications. https://opencontainers.org/